



VINUNIVERSITY

Lecture 5: Java Constructors and Inheritance

Constructors, Constructor Calls, and Inheritance

Hieu Pham, College of Engineering & Computer Science, VinUniversity

March 04, 2025

Course announcement



Pham Huy Hieu AUTHOR | TEACHER

Created Mar 2 10:15am | Posted Mar 2 10:15am

Class Relocation Announcement | Thursday, March 6 | Room C202

Dear Students,

Please be informed that our class **next Thursday, March 6**, will be moved from **Room E402** to **Room C202**. This change applies only to this day, and we will return to **Room E402** as usual afterward. Kindly arrive at the new location on time. Thank you!

If you have any questions, feel free to reach out.

Best regards,

Hieu Pham

Reply

Course announcement



Welcome to
VINUNIVERSITY
University of Excellence

Dear beloved colleagues,
We would like to introduce our newcomers who join VinUniversity since March 2025.

1. **Dr. Sungdeok (Steve) Cha – Teaching-focused Professor (Computer Science) - College of Engineering and Computer Science (sungdeok.c@vinuni.edu.vn)**



Software (CEEDS).

Dr. Sungdeok (Steve) Cha received B.S., M.S., and Ph.D. degrees in Information and Computer Science from University of California, Irvine. He joined the computer science department at Korea Advanced Institute of Science and Technology (KAIST) in Daejeon, Korea. While serving as a professor at KAIST, he was the founding director of Defense Software Research Center, funded by Korean government.

From 2008 to 2025, he was a professor at Korea University in Seoul, Korea in the Computer Science and Engineering department. As of March 2025, he is an emeritus professor. While at Korea University, he was the director of the Center for Engineering and Education of Dependable

Course announcement

Term Project Instructions

Item	Assessment Method	Weighting
1. Class Attendance, Quiz & Presentation	Online Quizzes and Class Presentation	10%
2. Weekly Laboratory	Hands-on Practice	20%
3. Term Project	Group Project (Program, Report, and Presentation)	30%
4. Midterm Exam	Written & Hands-on	20%
5. Final Exam	Written & Hands-on	20%
Total		100%

The sooner
the better!



Course announcement

Term Project Objectives

In general, the goal of an OOP term project is to apply the principles of OOP to design and implement a software solution to a given problem or scenario.

- Application of OOP principles
- Showing your problem-solving ability
- Practice software design (e.g. modularity and reusability of code)
- Improve your implementation skills
- Practice testing and debugging
- Documentation (project requirements, design specifications, class diagrams, and user manuals)
- Team collaboration (foster teamwork and collaboration skills)
- Communication skills (Improve your presentation skills through project presentations at the end of the course)

Course announcement

Term Project: Topic and Requirements

Topic: Two options.

- You can **propose** your idea/topic and then the instructors will validate it.
- We **recommend** a list of projects and then your team will choose one.

Schedule and Deliverables

- Form teams of up to **three** people and **topic selection**. Please fill out the team registration form by the deadline (TBD).
- Submit **a proposal** about what content will be covered in the group project (TBD).
- Submit your **report & presentation** (TBD).

Course announcement

Term Project: Topic and Requirements

Notes

- If you do not want to work with a team, we accept that.
- Feel free to pursue your own ideas. If you do not have any proposals, we will assign one to your team. Please do not select the topics that are already taken.
- All questions are welcome. Our responsibility is to help you to form good projects, track your progress and support during the time of conducting your project.

LECTURE 04 (Revisit)

Can someone summarize the key takeaway from our last lecture?

LECTURE 04 (Revisit)

Last lecture we have covered:

- Introduction to variables in Java (i.e., class variables, instance variables, and local variables).
- Abstract classes and methods (how does Java implement data abstraction).
- Java exceptions and catching exceptions (how does Java handle runtime errors gracefully)
- Modular programming (how to improve maintainability, readability and scalability of your Java programs)

LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand and use **constructors** in Java.
- understand and implement **inheritance types**, such as single, multi-level, and hierarchical inheritance.
- understand how to use **interface** to implement hybrid and multiple inheritance in Java.

JAVA CONSTRUCTORS

1. What they are?
2. What will they be used for?
3. And how are they used in Java?

Motivation example: A typical example of a method in Java

- A method is a “block of code” that performs a specific task and is called when needed.
- Here, `add(int a, int b)` is a method that takes two numbers and returns their sum.
- Called using `calc.add(5, 3)`

```
1 // Lecture 05 on Constructor in Java
2 // Example
3
4 class Calculator {
5     // Method to add two numbers
6     int add(int a, int b) {
7         return a + b;
8     }
9
10    public static void main(String[] args) {
11        Calculator calc = new Calculator(); // Calling the constructor (default one)
12        int result = calc.add(5, 3); // Calling the method
13        System.out.println("Sum: " + result);
14    }
15 }
16 // Sum: 8
```

A typical example of a method in Java

- A method is a block of code that performs a specific task and is called when needed.
- Here, `add(int a, int b)` is a method that takes two numbers and returns their sum.
- It is called using `calc.add(5, 3)`.

```
1 // Lecture 05 on Constructor in Java
2 // Example
3
4 class Calculator {
5     // Method to add two numbers
6     int add(int a, int b) {
7         return a + b;
8     }
9
10    public static void main(String[] args) {
11        Calculator calc = new Calculator(); // Calling the constructor (default one)
12        int result = calc.add(5, 3); // Calling the method
13        System.out.println("Sum: " + result);
14    }
15 }
16 // Sum: 8
```

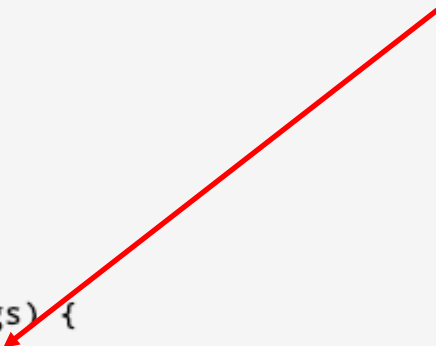
Question: Where is the object and how do you initialize it?

A typical example of a method in Java

- A method is a block of code that performs a specific task and is called when needed.
- Here, `add(int a, int b)` is a method that takes two numbers and returns their sum.
- It is called using `calc.add(5, 3)`.

```
1 // Lecture 05 on Constructor in Java
2 // Example
3
4 class Calculator {
5     // Method to add two numbers
6     int add(int a, int b) {
7         return a + b;
8     }
9
10    public static void main(String[] args) {
11        Calculator calc = new Calculator(); // Calling the constructor (default one)
12        int result = calc.add(5, 3); // Calling the method
13        System.out.println("Sum: " + result);
14    }
15 }
16 // Sum: 8
```

Call a constructor here!



How to initialize objects?

JAVA CONSTRUCTORS

What is a Java constructor?

- A **constructor** in Java is a special method that is used to **initialize** objects.
- **Every class has a constructor**, and it is called when an instance of the class is created.
- If no constructor is provided in a class, the Java compiler (Javac) creates **a default constructor** for that class.
- Java **automatically calls** that constructor when an object is created.

JAVA CONSTRUCTORS

- Note that the **constructor name must match the class name**, and it cannot have a return type (like **void**).
- The constructor is called when the object is created.

syntax

```
class ClassName {  
    ClassName() {  
    }  
}
```

The name of the constructor is the same as the name of the class.

This syntax shows you how to create a constructor in a class!

JAVA CONSTRUCTORS: Example

```
1 // Lecture 05 on Constructor in Java
2 // Example
3
4 class Person {
5     String name;
6
7     // Constructor to initialize name
8     Person(String personName) {
9         name = personName;
10    }
11
12    void display() {
13        System.out.println("Name: " + name);
14    }
15
16    public static void main(String[] args) {
17        Person p1 = new Person("Alice"); // Calling the constructor
18        p1.display();
19    }
20 }
21
22 // Name: Alice
```

- Here, **Person()** is a constructor. It has the same name as that of the class.
- It initializes the name variable.
- When **new Person("Alice")** is called, the constructor sets name = "Alice".
- Unlike methods, constructors do **not** have a return type.

TYPES OF JAVA CONSTRUCTORS

- A Java constructor may or may not have any parameters (arguments). If a constructor does not accept any parameters, it is known as a **no-argument constructor**.
- A Java constructor can also accept one or more parameters. Such constructors are known as **parameterized constructors** (constructors with parameters).

NO ARGUMENT CONSTRUCTOR

A Java constructor may or may not have any parameters (arguments). If a constructor does not accept any parameters, it is known as a **no-argument constructor**.

```
class Book{  
    //creating a default constructor  
    Book(){  
        System.out.println("Book is created.");  
    }  
}
```

without an
argument

```
public class MyJavaConstructors {  
  
    public static void main(String[] args) {  
        Book myBook = new Book();  
    }  
}
```

Book is created.

constructing an
object

PARAMETERIZED CONSTRUCTOR

A Java constructor can also accept one or more parameters. Such constructors are known as **parameterized constructors** (constructors with parameters).

```
class Main {  
  
    String languages;  
  
    // constructor accepting single value  
    Main(String lang) {  
        languages = lang;  
        System.out.println(languages + " Programming Language");  
    }  
  
    public static void main(String[] args) {  
  
        // call constructor by passing a single value  
        Main obj1 = new Main("Java");  
        Main obj2 = new Main("Python");  
        Main obj3 = new Main("C");  
    }  
}
```

In this example, we have created a constructor named **Main(String lang)** that takes a single parameter.

PARAMETERIZED CONSTRUCTOR

```
class Car{  
    int year;  
    String brand;  
    int price;  
  
    Car(int yr, String br){  
        year = yr;  
        brand = br;  
    }  
  
    void displayInfo() {  
        System.out.println("This " + brand + " car is manufactured in " + year);  
    }  
}
```

Constructor
with two
arguments

```
public class MyJavaConstructors {  
  
    public static void main(String[] args) {  
  
        Car myCar = new Car(2020, "VinFast");  
        myCar.displayInfo();  
    }  
}
```

DEFAULT CONSTRUCTOR

- When there is no explicit declaration of a constructor for any java class, the compiler creates a **default constructor** for that class which does not have any return type.
- A default constructor is a **no-argument** constructor.
- The default constructor **cannot be seen** in the source code (the .java file) as it is inserted during compilation and present in the bytecode(.class file).

DEFAULT CONSTRUCTOR

When there is no explicit declaration of a constructor for any java class, the compiler creates a **default constructor** for that class which

```
public class Student {  
    String firstName;  
    String lastName;  
    int age;  
  
    public static void main(String args[]) {  
        Student myStudent = new Student();  
  
        myStudent.firstName = "Ihechikara";  
        myStudent.lastName = "Abba";  
        myStudent.age = 100;  
  
        System.out.println(myStudent.age);  
        //100  
  
        System.out.println(myStudent.firstName);  
        //Ihechikara  
    }  
}
```

The program did not define any constructor before creating **myStudent** to initialize the attributes created in the class. The compiler will create **an empty constructor** but you will not see this constructor anywhere in the code.

Source: <https://www.freecodecamp.org/news/default-constructor-in-java/>

CONSTRUCTOR OVERLOADING

“Constructor overloading allows a class to have multiple constructors with the same name but different parameter lists, enabling flexible object creation with varying initializations.”

CONSTRUCTOR OVERLOADING

Having more than one constructor with different parameter lists

```
class Car{
    int year;
    String brand, color;
    int price;

    Car(String col){
        color = col;
        System.out.println("Car is created.");
    }

    Car(int yr, String br){
        year = yr;
        brand = br;
    }

    void displayInfo() {
        System.out.println("This " + brand + " car is manufactured in " + year);
    }

    void displayColor() {
        System.out.println("Your car is " + color);
    }
}
```

Each constructor must have its own unique parameter list.

```
public class MyJavaConstructors {

    public static void main(String[] args) {

        Car firstCar = new Car("Red");
        firstCar.displayColor();

        Car secondCar = new Car(2020, "VinFast");
        secondCar.displayInfo();
    }
}
```

```
Car is created.
Your car is Red
This VinFast car is manufactured in 2020
```

Class Practice

Java Constructor vs. Java Methods

What are the differences between Java constructor and Java methods?

Feature	Method	Constructor
Purpose	?	?
Name	?	?
Return type	?	?
Invocation (how it will be called)	?	?

Constructors vs. Methods

How Java Constructors are Different From Java Methods?

Feature	Method	Constructor
Purpose	Performs an action	Initializes an object
Name	Can be any valid identifier	Same as class name
Return Type	Has a return type (e.g., <code>int</code> , <code>void</code>)	No return type
Invocation	Called explicitly	Called automatically when an object is created

INHERITANCE

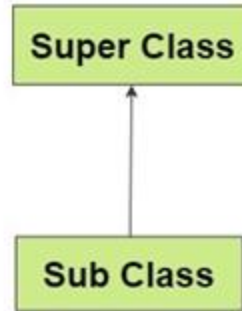
Types of Inheritance in Java, and Constructor
Chaining: `this()` & `super()`

INHERITANCE

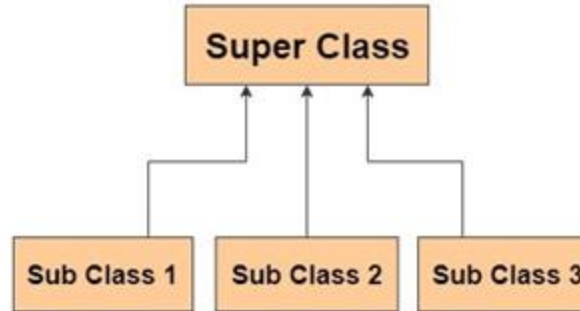
- Inheritance allows a programmer to handle a group of similar **objects in a uniform way**. It aims to minimize code duplication.
- **WARNING!** If inheritance is not utilized correctly, it can cause the code unmaintainable and even cause bugs in the long run.

TYPES OF INHERITANCE IN JAVA (REVISIT)

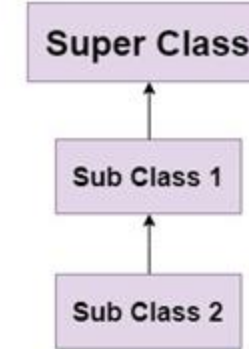
Single Inheritance



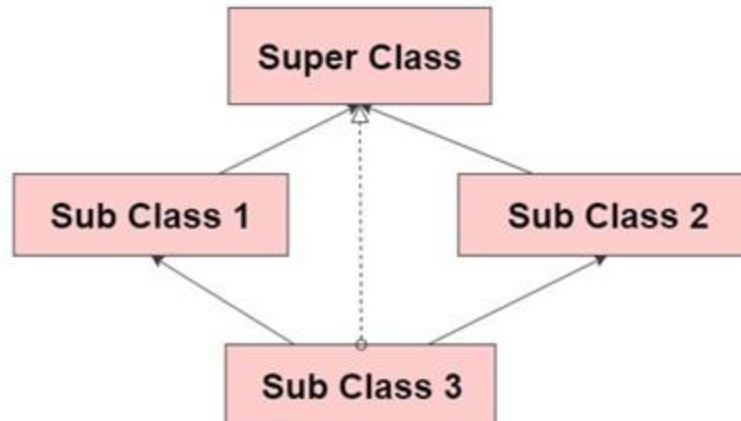
Hierarchical Inheritance



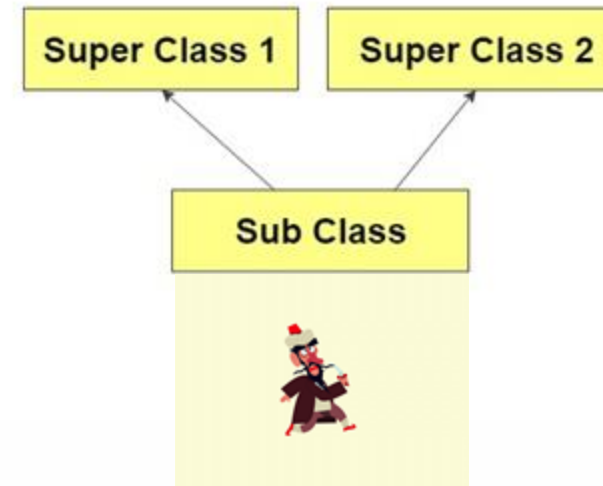
MultiLevel Inheritance



Hybrid Inheritance



Multiple Inheritance



[source]

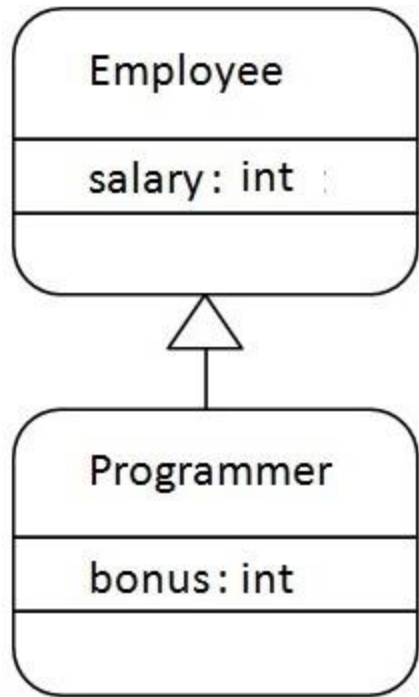
INHERITANCE IN JAVA

- Syntax:

```
class Subclass-name extends Superclass-name {  
    //methods and fields  
}
```

- Keyword: **extends**
- A class which is **inherited** is called a **super class** (or base class or parent class), and the new class is called **sub class** (derived class or child class).

SINGLE INHERITANCE EXAMPLE



super class

sub class

```
class Employee {
    int salary;
}
```

```
class Programmer extends Employee {
    int bonus;
}
```

```
public class MyInheritance {

    public static void main(String[] args) {
        Programmer p = new Programmer();
        p.bonus = 5000;
        p.salary = 100000;
        System.out.println(p.bonus + p.salary);
    }
}
```


MULTI-LEVEL INHERITANCE EXAMPLE

```
class Employee {  
    int salary;  
    String name;  
  
    Employee(){  
        System.out.println("New employee created.");  
    }  
  
    // parameterized constructor  
    Employee(String name){  
        this.name = name;  
        System.out.println("Welcome " + name);  
    }  
}
```

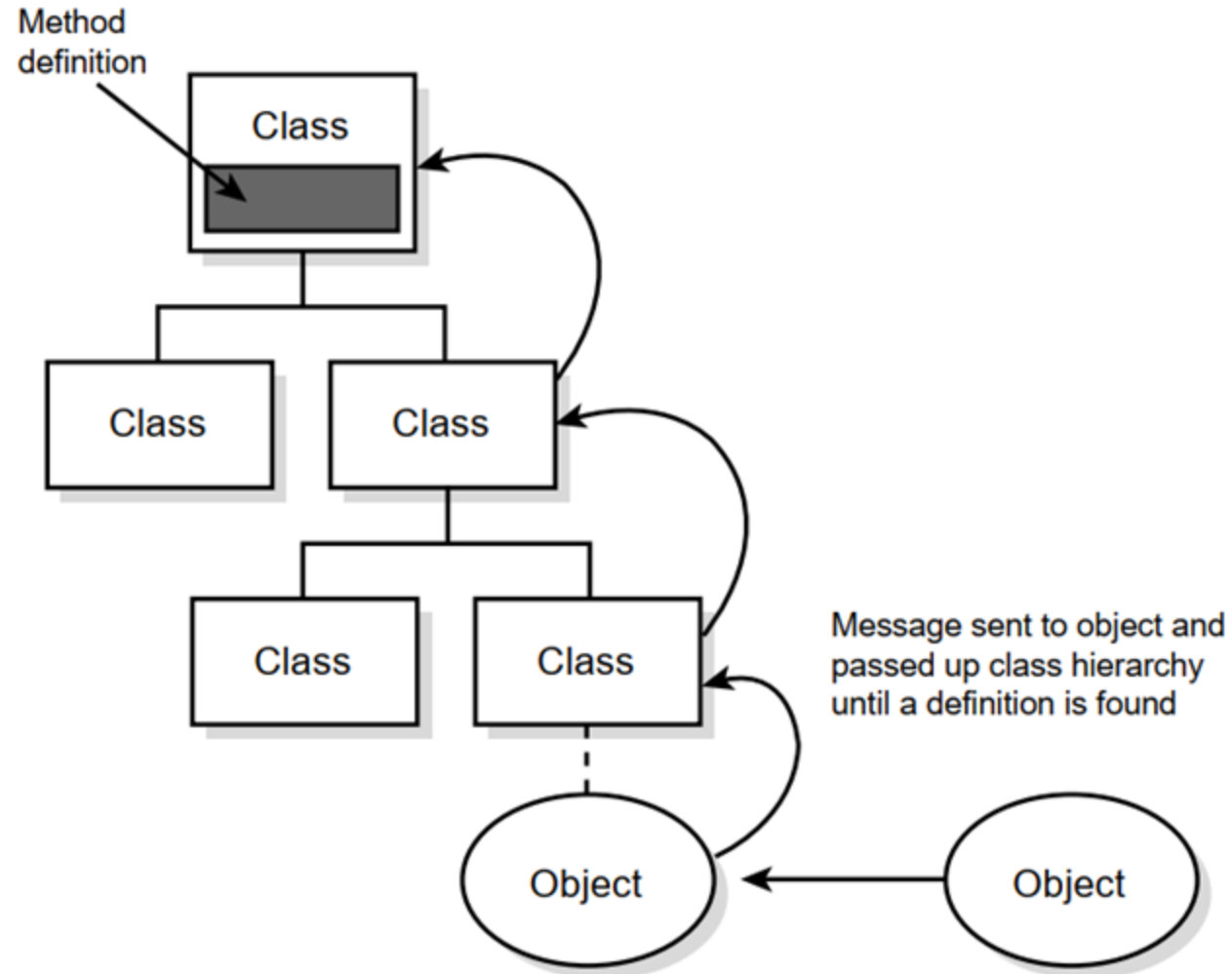
super class

sub class level 1

```
class Programmer extends Employee {  
    int bonus;  
  
    Programmer(String name){  
        super(name);  
    }  
  
    void displayDept() {  
        System.out.println("IT Department");  
    }  
}
```

```
class Trainee extends Programmer{  
    int hour;  
  
    Trainee(String name) {  
        super(name);  
    }  
}  
  
public class MyInheritance {  
  
    public static void main(String[] args) {  
        Programmer p = new Programmer("Ali");  
        p.bonus = 5000;  
        p.salary = 100000;  
        System.out.println(p.bonus + p.salary);  
  
        Trainee t = new Trainee("Aladdin");  
        t.bonus = 500;  
        t.salary = 100;  
        t.hour = 10;  
        System.out.println(t.bonus + t.salary * t.hour);  
        t.displayDept();  
    }  
}
```

HOW METHODS ARE LOCATED



CONSTRUCTOR CHAINING TO OTHER CLASS

```
class Employee {  
    int salary;  
  
    Employee(){  
        System.out.println("New employee created.");  
    }  
}
```

```
class Programmer extends Employee {  
    int bonus;  
  
    Programmer(){  
        super();  
    }  
}
```

Used to invoke a
superclass's constructor

In Java, `super()` is a special call that refers to the constructor of the immediate parent class.

```
public class MyInheritance {  
  
    public static void main(String[] args) {  
        Programmer p = new Programmer();  
        p.bonus = 5000;  
        p.salary = 100000;  
        System.out.println(p.bonus + p.salary);  
    }  
}
```

```
New employee created.  
105000
```

CONSTRUCTOR CHAINING TO OTHER CLASS

```
class Employee {  
    int salary;  
    String name;  
  
    Employee(){  
        System.out.println("New employee created.");  
    }  
  
    // parameterized constructor  
    Employee(String name){  
        this.name = name;  
        System.out.println("Welcome " + name);  
    }  
}
```

```
class Programmer extends Employee {  
    int bonus;  
  
    Programmer(String name){  
        //super(); -> to invoke default constructor  
        super(name);  
    }  
}
```

Used to invoke a superclass's parameterized constructor

```
public class MyInheritance {  
  
    public static void main(String[] args) {  
        Programmer p = new Programmer("Ali");  
        p.bonus = 5000;  
        p.salary = 100000;  
        System.out.println(p.bonus + p.salary);  
    }  
}
```

```
Welcome Ali  
105000
```

HIERARCHICAL INHERITANCE EXAMPLE

```
class Employee {  
    int salary;  
    String name;  
  
    Employee(){  
        System.out.println("New employee created.");  
    }  
  
    // parameterized constructor  
    Employee(String name){  
        this.name = name;  
        System.out.println("Welcome " + name);  
    }  
}
```

same super class

```
class Programmer extends Employee {  
    int bonus;  
  
    Programmer(String name){  
        super(name);  
    }  
  
    void displayDept() {  
        System.out.println("IT Department");  
    }  
}
```

```
// hierarchical inheritance  
class HumanResource extends Employee {  
    HumanResource() {  
        System.out.println("HR personnel");  
    }  
  
    void displayLocation() {  
        System.out.println("Main Branch");  
    }  
}  
  
public class MyInheritance {  
  
    public static void main(String[] args) {  
        Programmer p = new Programmer("Ali");  
        p.bonus = 5000;  
        p.salary = 100000;  
        System.out.println(p.bonus + p.salary);  
  
        HumanResource h = new HumanResource();  
        h.displayLocation(); h.displayDept();  
    }  
}
```

MULTIPLE INHERITANCE EXAMPLE

```
interface SuperClass1 {  
    void Read();  
}
```

multiple inheritance is supported
through **interface** only

```
interface SuperClass2 {  
    void Write();  
}
```

```
class SubClass implements SuperClass1, SuperClass2 {  
  
    public void Read() {  
        System.out.println("Reading...");  
    }  
  
    public void Write() {  
        System.out.println("Writing...");  
    }  
}
```

```
public class MyHybridMultipleInheritance {  
  
    public static void main(String[] args) {  
  
        SubClass s = new SubClass();  
        s.Read();  
        s.Write();  
    }  
}
```

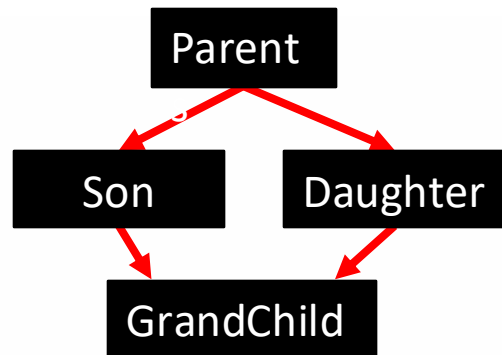
```
Reading...  
Writing...
```


HYBRID INHERITANCE EXAMPLE

```
class Parents {  
    public void showFamily() {  
        System.out.println("showFamily()->Parents class");  
    }  
}
```

```
interface Son {  
    public void showParents();  
}
```

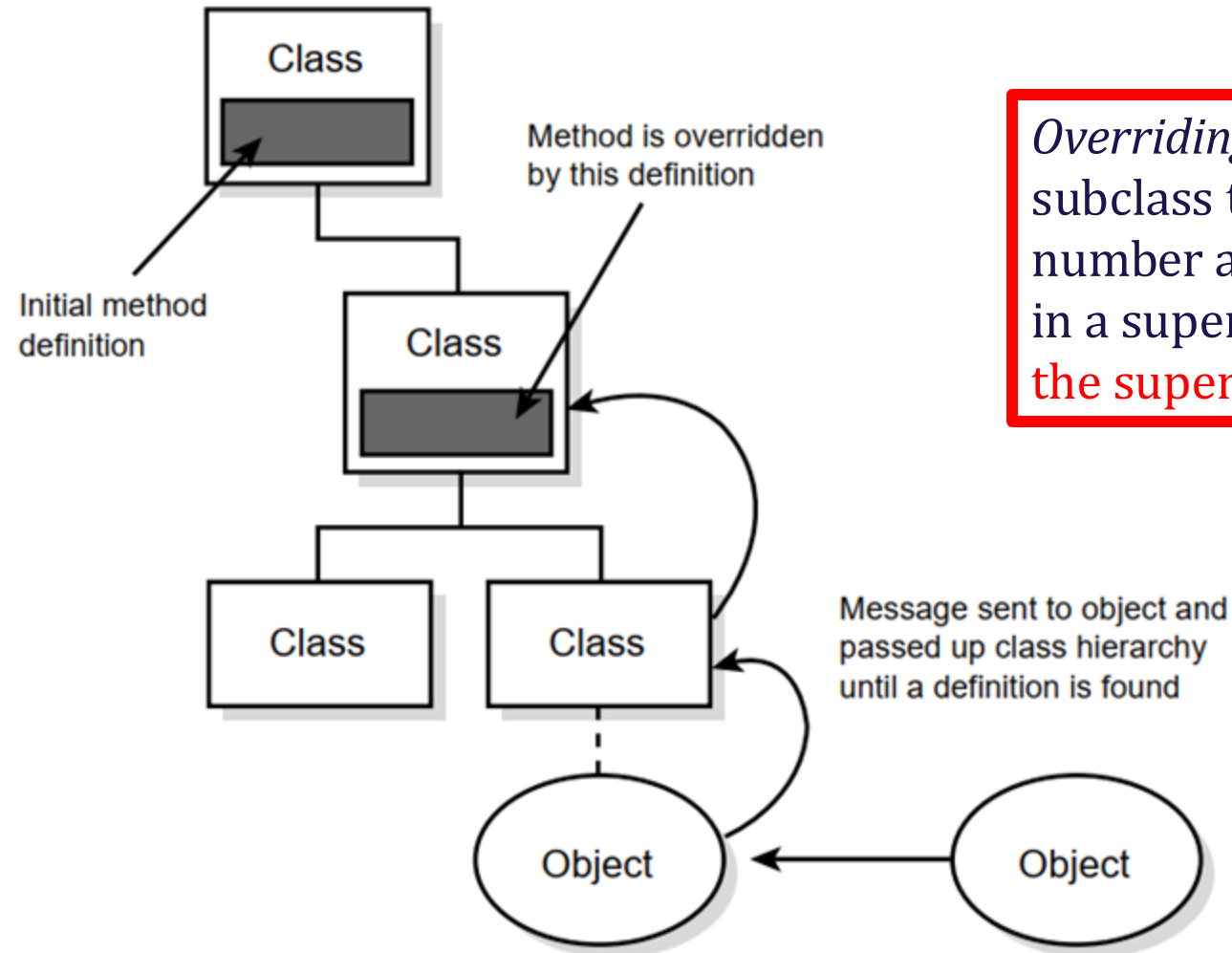
```
interface Daughter {  
    public void showParents();  
}
```



```
class GrandChild extends Parents implements Son, Daughter {  
    public void showParents() {  
        System.out.println("showParents() implementation");  
    }  
  
    public void showFamily() {  
        System.out.println("showFamily()->GrandChild class");  
    }  
}  
  
public class MyHybridInheritance {  
  
    public static void main(String[] args) {  
        GrandChild d = new GrandChild();  
        d.showFamily();  
        d.showParents();  
    }  
}
```

Output
?

OVERRIDING METHODS



Overriding a method is creating a method in a subclass that has the same signature (name, number and type of arguments) as a method in a superclass. **That new method then hides the superclass's method.**

HOMEWORK

Hybrid and Multiple Inheritance

Find extra examples (sample code) for each of the following types of inheritance in Java (by using the interface):

1. Hybrid Inheritance
2. Multiple Inheritance

Question: Why is it not possible to implement the above inheritances in Java without using the interface?

Submit your answer on Canvas before next Tuesday.

THANK YOU!